UNIVERSITY EVENTS

Team No.10

สมาชิกกลุ่ม

นางสาวกมลชนก เรืองแสน (Product Owner : PO)

พชรพล เชิดชู (Scrum Master : SM)

นายทิษณุ กลิ่นกำธรกุล (Developer)

นายรพีพัฒน์ ทับทอง (QA / DevOps)

นายเทพเมธี ศรีพล (QA / DevOps)

Project Charter

> โครงการนี้เป็นการพัฒนาแพลตฟอร์มออนไลน์เพื่อเปลี่ยนการจัดการกิจกรรมในมหาวิทยาลัยจากรูปแบบเอกสารมาเป็น ระบบดิจิทัลแบบครบวงจร ตั้งแต่การสร้างกิจกรรม การลงชื่อเข้าร่วม ไปจนถึงการสรุปผลผ่าน Dashboard

เป้าหมายหลัก

- สร้าง & จัดการ: ผู้จัดสามารถตั้งค่ากิจกรรมและจำกัดจำนวนคนได้แม่นยำ 100%
- สะดวก & รวดเร็ว: นักศึกษาสมัครและเช็กชื่อผ่านระบบออนไลน์ได้ทันที
- โปร่งใส & ตรวจสอบได้: มีรายงานสถิติที่ดูง่ายและถูกต้อง

การบริหารความเสี่ยงที่สำคัญ

ทีมให้ความสำคัญกับ "ความถูกต้องของข้อมูล" และ "ประสบการณ์ผู้ใช้" โดยมีแนวทางรับมือดังนี้

ระบบผิดพลาด: แก้ไขโดยการทำ Unit/Functional Testing อย่างเข้มงวด

การใช้งานยาก: แก้ไขโดยการออกแบบ UI/UX ให้เป็นมิตรกับผู้ใช้

เวลาไม่พอ: แก้ไขโดยการยึดตาม Milestone ที่ Scrum Master กำหนดอย่างเคร่งครัด

ปัญหาที่ระบบแก้

เป้าหมายหลัก 3 ข้อ

- สร้างระบบสำหรับจัดการกิจกรรมตั้งแต่สร้างกิจกรรมจนถึงอนุมัติ
- ลดเวลาการดำเนินงานของขั้นตอนสมัครและอนุมัติด้วย workflow ที่ชัดเจน
- เพิ่มความโปร่งใส โดยให้ผู้ใช้ติดตามสถานะได้แบบเรียลไทม์

KPI ความสำเร็จ

≥ 95%	อัตราความสำเร็จการใช้งานฟังก์ชันหลัก (Login, สมัครกิจกรรม, อนุมัติ)
≤ 2 วินาที	เวลาเฉลี่ยในการตอบสนอง API หลัก
≥ 90%	อัตราการผ่านชุดทดสอบอัตโนมัติ

Requirement Planning Summary

Scope

- พัฒนาระบบจัดการกิจกรรมมหาวิทยาลัยแบบเว็บแอป
- ครอบคลุมการจัดการผู้ใช้ กิจกรรม การสมัคร และการอนุมัติ
- รองรับบทบาทหลัก Student, Organizer, Approver, Admin

Timeline

- Phase 1: Requirement Analysis และ Planning
- Phase 2: System Design และ Development
- Phase 3: Testing, Validation และ Final Delivery

Stakeholders

- นักศึกษา (ผู้สมัครกิจกรรม)
- ผู้จัดกิจกรรม (สร้าง/แก้ไขกิจกรรม)
- ผู้อนุมัติ (ตรวจสอบและอนุมัติคำขอ)
- ผู้ดูแลระบบ (จัดการสิทธิ์และข้อมูลระบบ)
- คณะ/มหาวิทยาลัย (หน่วยงานเจ้าของระบบ)

Success Criteria

- ฟังก์ชันหลักทำงานครบตาม SRS (Auth, Event, Registration, Approval)
- ผู้ใช้แต่ละบทบาทใช้งาน workflow ได้ถูกต้อง
- ระบบผ่านการทดสอบตามแผนและมีเอกสารประกอบครบ
- มีการจัดการความเสี่ยงและติดตามสถานะโครงการอย่างต่อเนื่อง

User Stories

รหัส	User Story	Story Point	ลำดับความสำคัญ
1	การสมัครผู้ใช้	5	สูง
2	การเข้าสู่ระบบ	3	สูง
3	การสร้างกิจกรรม	8	สูง
4	การแก้ไขกิจกรรม	5	ปานกลาง
5	การแสดงรายการกิจกรรม	5	สูง
6	การสมัครเข้าร่วมกิจกรรม	8	สูง
7	การจำกัดจำนวนผู้เข้าร่วม	5	สูง
8	การเช็กชื่อเข้าร่วมกิจกรรม	8	สูง
9	การดูประวัติการเข้าร่วมกิจกรรม	5	ปานกลาง
10	รายงานสรุปผลและ Dashboard	8	ปานกลาง
11	อนุมัติกิจกรรม	5	สูง
12	ประเมินความพึงพอใจ	5	ปานกลาง
13	ค้นหาและกรองกิจกรรม	5	สูง

รวม Story Point: 75

แสดงความต้องการของผู้ใช้งานในระบบ
ระบุฟีเจอร์หลัก เช่น

- สมัครสมาชิก / เข้าสู่ระบบ
- สร้างและจัดการกิจกรรม
- สมัครเข้าร่วมกิจกรรม
- ดูรายงานและ Dashboard

ใช้ Story Point เพื่อประเมินความยากของแต่ละงาน
มีการกำหนด ลำดับความสำคัญ (สูง / ปานกลาง)
รวมทั้งหมด 75 Story Points
ใช้สำหรับวางแผนและจัดลำดับการพัฒนา

Risk Snapshot

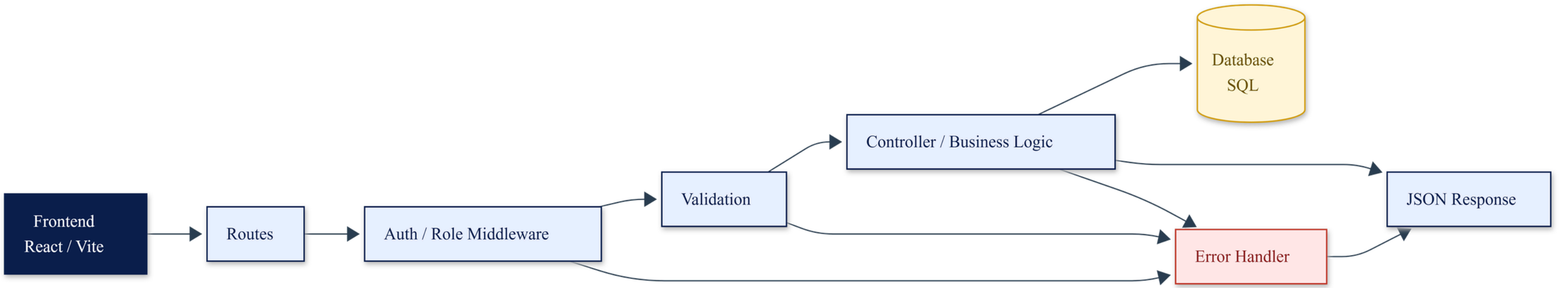
Heatmap (ความเสี่ยงโครงการ)

- ผลกระทบสูง / โอกาสเกิดสูง: R1 งานล่าช้าตามแผน
- ผลกระทบสูง / โอกาสเกิดต่ำ: R2 การกำหนดสิทธิ์เข้าถึงผิดพลาด
- ผลกระทบต่ำ / โอกาสเกิดสูง: R3 ความต้องการเปลี่ยนระหว่างพัฒนา
- ผลกระทบต่ำ / โอกาสเกิดต่ำ: R4 ความไม่สม่ำเสมอของ UI เล็กน้อย

ความเสี่ยงหลัก 3 อันดับ + แผนรับมือ

- R1 งานล่าช้าตามแผน: แบ่งงานเป็น Sprint ระยะสั้น และติดตาม Milestone รายสัปดาห์
- R2 การกำหนดสิทธิ์เข้าถึงผิดพลาด: ใช้ Middleware สำหรับยืนยันตัวตน/ตรวจสิทธิ์ และทดสอบ Integration ด้านสิทธิ์การเข้าถึง
- R3 ความต้องการเปลี่ยนระหว่างพัฒนา: บันทึก Change Log และกำหนดขอบเขตงานคงที่ในแต่ละ Sprint

Architecture & Data Flow



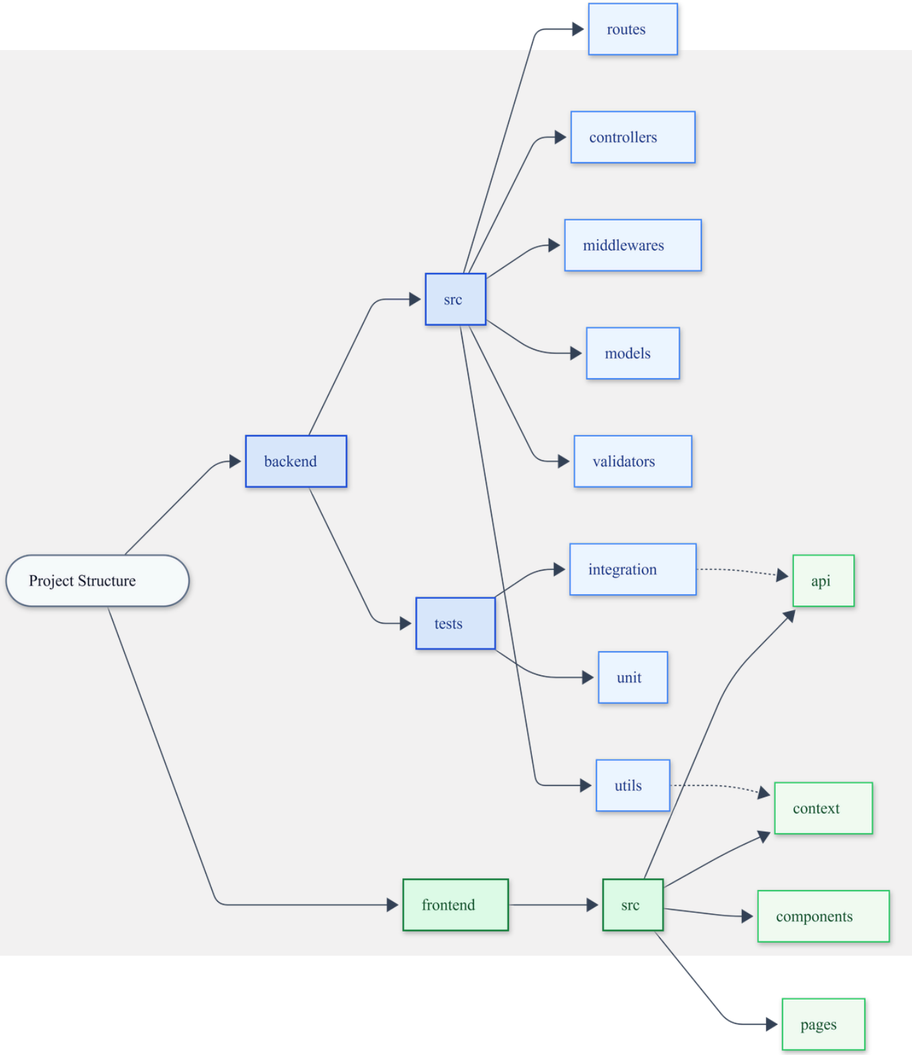
ระบบเริ่มจาก Frontend ส่งคำขอไป Backend จากนั้น Backend จะตรวจสอบสิทธิ์ผู้ใช้ ตรวจสอบความถูกต้องของข้อมูล แล้วค่อยประมวลผลกับฐานข้อมูล สุดท้ายส่งผลลัพธ์กลับเป็น JSON และถ้ามีข้อผิดพลาดจะถูกจัดการผ่าน error handler กลาง ทำให้ระบบปลอดภัย ตรวจสอบง่าย

Design Patterns + Code Skeleton

Pattern ที่เลือก

- MVC-like Separation แยกหน้าที่ Route, Controller, Model ชัดเจน ทำให้แก้ไขและทดสอบได้ง่าย
- Middleware Pattern รวมงานที่ใช้ซ้ำ เช่น Auth, Role Check, Error Handling ลดโค้ดซ้ำในแต่ละ endpoint
- Layered Flow (Route → Controller → Data Access)

ควบคุมลำดับการทำงานชัดเจน ลดผลกระทบเมื่อเพิ่มฟีเจอร์ใหม่

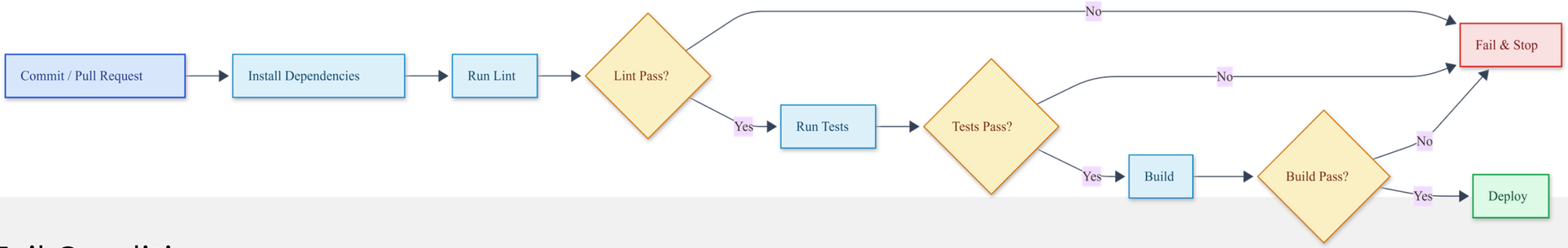


Testing Strategy & Coverage

ใช้แนวทาง Test Pyramid: เน้น Unit Tests เป็นฐาน และเสริม Integration Tests สำหรับ flow สำคัญ

- Unit Tests
  - ทดสอบ logic รายฟังก์ชัน/รายโมดูล เช่น validator, controller method, utility
  - รันเร็ว ใช้สำหรับตรวจ regression
- Integration Tests
  - ทดสอบการทำงานร่วมกันของ API route + middleware + controller + database flow
  - เน้น scenario สำคัญ เช่น login, create event, register, approve/reject

CI/CD



Pass/Fail Conditions

- Pass:

1. Lint ผ่านทั้งหมด (ไม่มี error ระดับบล็อก)
2. Test ผ่านทั้งหมด
3. Build สำเร็จ
4. Coverage ถึงเกณฑ์ที่ทีมกำหนด

- Fail:

5. มี lint error
6. มี test fail อย่างน้อย 1 เคส
7. Build ไม่สำเร็จ
8. Coverage ต่ำกว่าเกณฑ์ขั้นต่ำ
9. เมื่อ Fail ให้หยุดที่ stage นั้นทันที และไม่ deploy

Quality & Security

Code Quality Metrics ที่ใช้

- Test Coverage: Statements 94.2%, Functions 100%, Lines 94.2%, Branches 73.58%
- Test Result Stability: 69/69 tests ผ่าน
- Coding Standards Compliance: ใช้ ESLint และโครงสร้างแยกชั้น (routes/controllers/middlewares/validators)
- Maintainability: แยกโมดูลตามหน้าที่ ลดโค้ดซ้ำ และทำให้รีวิวโค้ดง่าย

Security assessment

- Authentication: ใช้ JWT สำหรับยืนยันตัวตน
- Authorization: ใช้ Role-based access control (RBAC) จำกัดสิทธิ์ตามบทบาท
- Input Validation: ตรวจสอบข้อมูลขาเข้าก่อนเข้าธุรกิจหลัก
- Centralized Error Handling: จัดการข้อผิดพลาดกลาง ลดการเปิดเผยข้อมูลระบบเกินจำเป็น
- Route Protection: endpoint สำคัญถูกป้องกันด้วย middleware

Team No.10

THANK YOU

06 April, 2026